

Jenkins CI-CD Demo — Node.js and Spring Boot

1. What Are We Building?

In this demo, we will build a complete CI/CD pipeline using **Jenkins**.

We will first implement the pipeline for a small **Node.js application** and then repeat the same idea with a **Spring Boot application**.

The complete flow will look like:

```
Developer
  ↓
Git
  ↓
GitHub
  ↓
Jenkins
  ↓
Checkout
  ↓
Install / Compile
  ↓
Lint / Quality Checks
  ↓
Automated Tests
  ↓
Build
  ↓
Create Artifact
  ↓
Deploy
  ↓
Smoke Test
  ↓
Running Application
```

For this demo, we are keeping the infrastructure simple.

```
Mac
→ Developer Workstation
```

```
Ubuntu VM
→ Jenkins Server
→ Jenkins Controller
→ Build Machine
```

We are **not** using a separate Jenkins agent machine.

The Jenkins Controller and the machine executing our builds are the same Ubuntu VM.

Part 1 — Setting Up Jenkins

2. Prepare the Ubuntu Machine

Update existing packages:

```
sudo apt update
sudo apt upgrade -y
```

Install a few basic utilities:

```
sudo apt install -y git curl wget fontconfig
sudo apt install -y libatomic1
```

Verify Git:

```
git --version
```

3. Install Java

Jenkins itself runs on Java.

We are also going to build a Spring Boot application later, so we install a **JDK** instead of only installing a JRE.

```
sudo apt install -y openjdk-21-jdk
```

Verify:

```
java -version
```

```
javac -version
```

Conceptually:

```
Java 21
├── Runs Jenkins
└── Compiles our Spring Boot application
```

4. Install Jenkins

Add the Jenkins Signing Key

Create a directory for repository keys:

```
sudo mkdir -p /etc/apt/keyrings
```

Download the Jenkins public signing key:

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
```

What does `wget` do?

`wget` downloads resources from the internet.

Example:

```
wget https://example.com/file.txt
```

The `-O` option tells `wget` where the downloaded file should be stored.

```
wget
→ Download something

-O
→ Save it using this filename/location
```

The `.asc` file contains an ASCII-armored representation of the public key.

5. Why Does Jenkins Need a Signing Key?

When we run:

```
sudo apt install jenkins
```

Ubuntu needs to make sure that the Jenkins package actually came from the Jenkins project and was not modified by somebody else.

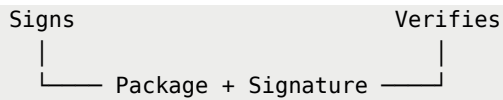
Jenkins signs its repository metadata using a **private signing key**.

```
Jenkins Project
Package / Repository Metadata
↓
Signed using
↓
Jenkins Private Key
↓
Digital Signature
```

The private key remains secret with Jenkins.

Our Ubuntu machine receives the corresponding **public key**.

```
Jenkins Private Key      Jenkins Public Key
|                         |
```



The public key allows Ubuntu to verify the signature.

It cannot be used to create valid Jenkins signatures.

We can inspect the downloaded public key:

```
cat /etc/apt/keyrings/jenkins-keyring.asc
```

It will look roughly like:

```
-----BEGIN PGP PUBLIC KEY BLOCK-----  
  
...  
  
-----END PGP PUBLIC KEY BLOCK-----
```

6. Add the Jenkins Repository

```
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]\  
https://pkg.jenkins.io/debian-stable binary/" | \  
sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
```

Update package metadata:

```
sudo apt update
```

Install Jenkins:

```
sudo apt install -y jenkins
```

Start Jenkins and configure it to start automatically:

```
sudo systemctl enable --now jenkins
```

Check its status:

```
sudo systemctl status jenkins
```

We should see:

```
active (running)
```

To stop Jenkins later:

```
sudo systemctl stop jenkins
```

7. Verify Jenkins Is Listening

Jenkins runs on port:

```
8080
```

Check whether something is listening there:

```
sudo ss -lntp | grep 8080
```

Understanding the Pipe |

The vertical bar is called a **pipe**.

```
Command 1 | Command 2
```

It sends the output of Command 1 directly into Command 2.

For example:

```
sudo ss -lntp | grep 8080
```

Here:

```
ss
↓
Shows listening network ports

|

grep 8080
↓
Filters the output and shows lines containing 8080
```

8. Access Jenkins From the Mac

Find the Ubuntu VM IP:

```
hostname -I
```

Example:

```
192.168.64.5
```

Then open from the Mac:

```
http://192.168.64.5:8080
```

If using Multipass, the VM IP can also be checked from the Mac:

```
multipass list
```

9. Unlock Jenkins

During the first startup, Jenkins asks for an initial administrator password.

Get it from Ubuntu:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy the generated value and paste it into the Jenkins UI.

Then choose:

Install suggested plugins

After installation, create the Jenkins administrator account.

10. Jenkins Controller, Node and Executor

Our current setup looks like:

```
Ubuntu VM
  ↓
Jenkins Process
  ↓
Jenkins Controller
  ↓
Built-In Node
  ↓
Executor
```

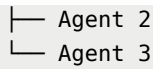
An **executor** is basically a slot that Jenkins can use to execute a job.

For this classroom demo:

```
Controller Machine
=
Build Machine
```

In a larger system we could instead have:

```
Jenkins Controller
  |
  └─ Agent 1
```



└─ Agent 2
└─ Agent 3

Different agents could execute different builds.

To inspect the current node:

Manage Jenkins → Nodes → Built-In Node

For this demo, having a couple of executors is sufficient.

11. Jenkins Plugins

Go to:

Manage Jenkins → Plugins

Useful plugins for our pipeline include:

- Git
- Pipeline
- Pipeline: Multibranch
- JUnit
- Pipeline Stage View
- NodeJS

Most are installed with the suggested plugin set.

For the Node.js project, we specifically need the **NodeJS plugin**.

The plugin allows Jenkins to install and expose Node.js and npm automatically during Pipeline execution.

12. Configure Node.js in Jenkins

Go to:

Manage Jenkins → Tools

Find:

NodeJS installations

Add a new installation:

```
Name :  
NodeJS-26
```

Enable:

```
Install automatically
```

Select the appropriate `26.x` version.

The name is important because our Jenkinsfile will refer to it:

```
tools {  
    nodejs 'NodeJS-26'  
}
```

13. Prepare Deployment Directories

Our applications will be deployed under:

```
/opt/cicd-demo
```

Create directories:

```
sudo mkdir -p /opt/cicd-demo/node/releases  
sudo mkdir -p /opt/cicd-demo/spring/releases
```

Jenkins runs as the Linux user:

```
jenkins
```

Give Jenkins ownership:

```
sudo chown -R jenkins:jenkins /opt/cicd-demo
```

Verify that Jenkins can write there:

```
sudo -u jenkins touch /opt/cicd-demo/test-file
```

Remove the test file:

```
sudo rm /opt/cicd-demo/test-file
```

Now Jenkins has permission to deploy applications into this directory.

Part 2 — Node.js CI/CD Demo

14. Node.js Application

Our application is deliberately simple.

It contains:

```
src/  
├─ math.js  
├─ app.js  
└─ server.js  
  
tests/  
├─ math.test.js  
└─ app.test.js
```

The idea is to keep the business logic separate from Express so that it can easily be unit tested.

15. Important npm Scripts

Inside `package.json`:

```
"scripts": {  
  "dev": "node src/server.js",  
  "start": "node dist/server.js",  
  "lint": "eslint src tests",  
  "test": "jest",  
  "test:ci": "JEST_JUNIT_OUTPUT_DIR=reports/junit JEST_JUNIT_OUTPUT_NAME=junit.xml jest --ci --coverage --reporters=default --reporters=jest-junit",  
  "build": "rm -rf dist && esbuild src/server.js --bundle --platform=node --packages=external --outfile=dist/server.js"  
}
```

These commands give us everything needed for CI:

```
npm run lint  
→ Code-quality checks  
  
npm run test:ci  
→ Automated tests + JUnit report  
  
npm run build  
→ Creates production build  
  
npm start  
→ Starts built application
```

16. Application Endpoints

The demo application contains a health endpoint:

```
GET /health
```

Example response:

```
{
  "status": "UP",
  "service": "jenkins-node-demo",
  "version": "dev"
}
```

And a simple addition API:

```
GET /api/add?a=20&b=22
```

Response:

```
{
  "result": 42
}
```

The application reads:

```
APP_VERSION
```

from an environment variable.

This becomes useful during deployment because Jenkins can inject its build number as the application version.

17. Automated Tests

The demo contains both:

```
Unit Tests
+
API Tests
```

Initially:

```
4 Unit Tests
+
3 API Tests
=
7 Automated Tests
```

Tests give Jenkins an automated way to decide whether the code should be allowed to move forward.

18. Linting

ESLint helps detect problems such as:

```
Unused variables
Undefined variables
Certain syntax/programming mistakes
```

This becomes one of our earliest CI quality gates.

19. `.gitignore`

Example:

```
node_modules/
dist/
artifacts/
coverage/
reports/
.DS_Store
```

Do **not** ignore:

```
package-lock.json
```

The lockfile should be committed.

Our CI pipeline uses:

```
npm ci
```

which relies on the committed lockfile.

20. Test Everything Manually First

A useful CI/CD rule is:

If you don't understand how to build the application manually, you shouldn't automate it yet.

Before writing a Jenkins pipeline, verify all commands yourself.

Lint

```
npm run lint
```

Tests

```
npm test
```

Expected:

```
7 passed
```

Build

```
npm run build
```

Check the output:

```
ls dist
```

Expected:

```
server.js
```

Start the built application

```
npm start
```

Check:

```
http://localhost:3000/health
```

And:

```
http://localhost:3000/api/add?a=20&b=22
```

Stop the application using:

```
Ctrl + C
```

21. Push the Project to GitHub

Create a repository:

```
jenkins-node-demo
```

Initialize Git:

```
git init  
git add .
```

```
git commit -m "Create Node CI/CD demo application"
git branch -M main
```

Add the GitHub remote:

```
git remote add origin https://github.com/YOUR_USERNAME/jenkins-node-demo.git
```

Push:

```
git push -u origin main
```

For the first demo, using a public repository keeps authentication from distracting us from Jenkins itself.

Part 3 — Jenkins Pipeline

22. Jenkinsfile

The `Jenkinsfile` describes the delivery process as code.

At the highest level:

```
pipeline
  ↓
agent
  ↓
tools
  ↓
options
  ↓
stages
```

A simplified view of our Node.js pipeline is:

```
pipeline {
```

```

agent any

tools {
    nodejs 'NodeJS-26'
}

options {
    skipDefaultCheckout(true)
    timestamps()
    disableConcurrentBuilds()
}

stages {

    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Environment') {
        steps {
            sh '''
                node --version
                npm --version
            '''
        }
    }

    stage('Install Dependencies') {
        steps {
            sh 'npm ci'
        }
    }

    stage('Lint') {

```

```

        steps {
            sh 'npm run lint'
        }
    }

    stage('Tests') {
        steps {
            sh 'npm run test:ci'
        }

        post {
            always {
                junit testResults: 'reports/junit/junit.xml',
                    allowEmptyResults: true
            }
        }
    }

    stage('Build') {
        steps {
            sh 'npm run build'
        }
    }

    stage('Package Artifact') {
        steps {
            sh '''
                rm -rf artifacts
                mkdir -p artifacts

                SHORT_SHA=$(git rev-parse --short HEAD)

                tar -czf\
                    "artifacts/node-demo-${BUILD_NUMBER}-${SHORT_SHA}.tar.gz"\

```

```

        dist\
        package.json\
        package-lock.json
    ...

    archiveArtifacts(
        artifacts: 'artifacts/*.tar.gz',
        fingerprint: true
    )
}
}

stage('Deploy') {

    when {
        branch 'main'
    }

    steps {
        // Deploy application
    }
}

stage('Smoke Test') {

    when {
        branch 'main'
    }

    steps {
        // Verify deployed application
    }
}

}

post {

```

```
        success {
            echo 'PIPELINE SUCCESSFUL'
        }

        failure {
            echo 'PIPELINE FAILED - deployment was blocked'
        }
    }
}
```

23. **agent any**

```
agent any
```

This tells Jenkins that the pipeline can run on any suitable available Jenkins agent. In our classroom setup, this will be the built-in node on the Jenkins VM.

24. Why **skipDefaultCheckout(true)** ?

Declarative Jenkins normally performs a source checkout automatically. We disable that:

```
skipDefaultCheckout(true)
```

because we want checkout to appear explicitly as its own stage:

```
stage('Checkout') {
    steps {
        checkout scm
    }
}
```

This makes the CI flow easier to understand.

25. What Does `checkout scm` Mean?

`SCM` stands for:

Source Control Management

Examples include:

```
GitHub
GitLab
Bitbucket
```

`checkout scm` means:

| Check out the source code associated with the current Jenkins job.

Conceptually:

```
GitHub Repository
  ↓
checkout scm
  ↓
Jenkins Workspace
```

After checkout, Jenkins has files such as:

```
src/
tests/
package.json
package-lock.json
Jenkinsfile
```

inside its workspace.

26. Why `disableConcurrentBuilds()` ?

```
disableConcurrentBuilds()
```

prevents multiple executions of the same job from running simultaneously.

Without it:

```
Build 10  
→ Deploying  
  
Build 11  
→ Deploying at the same time
```

Both builds could modify the same deployment environment.

Preventing this keeps deployment simpler and safer.

27. Why `npm ci` Instead of `npm install` ?

During development:

```
npm install
```

is commonly used when dependencies are being added or changed.

CI should instead perform a deterministic installation using the committed lockfile:

```
npm ci
```

Mental model:

```
Developer  
→ npm install  
→ Dependency changes  
→ package-lock.json updated  
  
CI  
→ npm ci  
→ Install exactly from lockfile
```

The goal is reproducibility.

28. Why Does Lint Run Before Tests?

Linting is generally cheaper than running an entire test suite.

Imagine:

```
Lint
→ 2 seconds

Tests
→ 2 minutes
```

If lint already fails, running expensive tests is unnecessary.

This follows a common CI design principle:

Fail Fast

Our sequence is therefore:

```
Install
↓
Lint
↓
Tests
↓
Build
```

29. JUnit Reports

Jest normally gives console output:

```
PASS
FAIL
7 passed
```

But Jenkins benefits from structured test information.

Using `jest-junit`, we generate:

```
reports/junit/junit.xml
```

Then Jenkins reads it:

```
junit testResults: 'reports/junit/junit.xml'
```

Now Jenkins knows:

```
How many tests ran?  
Which tests passed?  
Which tests failed?  
What were their names?  
How long did they take?
```

This is much more useful than Jenkins simply knowing:

```
Command exited with code 0
```

30. Why Create an Artifact?

A build should produce a clearly identifiable output.

The basic idea is:

```
Source Code  
  ↓  
Build  
  ↓  
Trusted Artifact
```

For our Node.js application we create something like:

```
node-demo-12-a73fe1.tar.gz
```

instead of relying on files that happen to exist somewhere inside the Jenkins workspace.

The artifact becomes the thing that moves toward deployment.

31. Jenkins **BUILD_NUMBER**

Jenkins automatically provides several environment variables.

One of them is:

```
BUILD_NUMBER
```

For example:

```
Build #1  
Build #2  
Build #3
```

During Build #3:

```
BUILD_NUMBER=3
```

So:

```
node-demo-${BUILD_NUMBER}.tar.gz
```

becomes:

```
node-demo-3.tar.gz
```

This gives us very simple artifact versioning.

32. `archiveArtifacts`

```
archiveArtifacts(  
    artifacts: 'artifacts/*.tar.gz',  
    fingerprint: true  
)
```

This tells Jenkins to store the generated artifact with the build.

Instead of:

```
Artifact exists somewhere in workspace
```

we now get:

```
Jenkins  
↓  
Build #3  
↓  
Artifacts  
↓  
node-demo-3-xxxxxx.tar.gz
```

In larger systems, dedicated artifact repositories such as **Nexus** or **Artifactory** can be used.

33. Deployment

The deployment stage follows a simple idea:

```
Deploy only from main  
↓  
Create release directory  
↓  
Extract new artifact
```

```
↓
Install production dependencies
↓
Stop previous application
↓
Point to new release
↓
Start new application
```

The release directory is based on the Jenkins build number:

```
/opt/cicd-demo/node/releases/BUILD_NUMBER
```

This gives each Jenkins build its own release directory.

34. Deploy Only From `main`

The pipeline contains:

```
when {
    branch 'main'
}
```

This creates an important distinction:

```
Feature Branch
=
Validate Code

Main Branch
=
Validate Code + Deploy
```

A successful feature branch does **not** automatically get permission to modify the deployment environment.

35. Smoke Testing

A successful deployment command does not guarantee that the application is actually working.

```
Deployment command succeeded
≠
Application works
```

So after deployment we perform a **smoke test**.

```
Deploy
  ↓
Start Application
  ↓
Call Health Endpoint
  ↓
Verify Application Responds
```

For the Node app:

```
curl -fsS http://127.0.0.1:3000/health
```

The pipeline retries the request for a short period because the application may take a moment to start.

It can also verify a functional endpoint:

```
curl -fsS "http://127.0.0.1:3000/api/add?a=20&b=22"
```

Part 4 — Multibranch Pipeline

36. What Is a Multibranch Pipeline?

Instead of creating a single Jenkins pipeline manually for one branch, Jenkins can discover branches containing a `Jenkinsfile`.

Create:

New Item → Multibranch Pipeline

Name:

```
jenkins-node-demo
```

Under Branch Sources:

```
Add Source  
→ Git
```

Repository:

```
https://github.com/YOUR_USERNAME/jenkins-node-demo.git
```

For a public repository:

```
Credentials: None
```

37. Repository Scanning

Jenkins periodically scans the repository and discovers branches containing a Jenkinsfile.

Conceptually:

```
Scanning repository...  
  ↓  
Found Jenkinsfile  
  ↓  
Found branch main  
  ↓  
Create Jenkins job for main  
  ↓  
Run pipeline
```

This is one of the major advantages of **Pipeline as Code**.

We did not manually create buttons for:

```
Checkout
Lint
Test
Build
Deploy
```

Jenkins reads the delivery process from the repository itself.

38. Pipeline Stages

Our Node pipeline contains:

```
Checkout
  ↓
Environment
  ↓
Install Dependencies
  ↓
Lint
  ↓
Tests
  ↓
Build
  ↓
Package Artifact
  ↓
Deploy
  ↓
Smoke Test
```

The Jenkins Console Output shows the commands executed during each stage.

For example:

```
node --version
npm ci
npm run lint
```

```
npm run test:ci  
npm run build
```

39. Verify the Artifact

Open a completed Jenkins build.

Under **Artifacts**, we should see something similar to:

```
node-demo-1-7bd29e.tar.gz
```

That file is the actual output produced by our CI build.

Conceptually:

```
Source Code  
  ↓  
Jenkins Build  
  ↓  
Versioned Artifact
```

40. Verify the Deployment

From the Mac:

```
curl http://VM_IP:3000/health
```

Example response:

```
{  
  "status": "UP",  
  "service": "jenkins-node-demo",  
  "version": "1"  
}
```

Notice:

```
version
=
Jenkins BUILD_NUMBER
```

This proves that the running application came from a particular Jenkins build.

Test the API:

```
curl "http://VM_IP:3000/api/add?a=20&b=22"
```

Expected:

```
{
  "result": 42
}
```

We have now completed:

```
Git
↓
GitHub
↓
Jenkins
↓
Lint
↓
Tests
↓
Build
↓
Artifact
↓
Deployment
↓
Running Application
```

Part 5 — Breaking the Pipeline

A CI system becomes much more useful when something goes wrong.

The important idea is:

Bad code should lose permission to move forward.

41. Demo 1 — Lint Failure

Create a new branch:

```
git checkout -b feature/ci-demo
```

Add an unused variable:

```
const uselessVariable = 100;
```

Commit and push:

```
git add .  
git commit -m "Introduce lint failure"  
git push -u origin feature/ci-demo
```

Jenkins discovers the new branch and starts its pipeline.

Expected result:

Checkout	✓
Environment	✓
Install Dependencies	✓
Lint	✗
Tests	SKIPPED
Build	SKIPPED
Package	SKIPPED
Deploy	SKIPPED

This demonstrates a **CI gate**.

```
Bad Code
  ↓
Quality Gate Fails
  ↓
Pipeline Stops
  ↓
No Artifact
  ↓
No Deployment
```

42. Fix the Lint Failure

Remove the unused variable.

Commit:

```
git add .
git commit -m "Fix lint issue"
git push
```

The next pipeline should now pass lint:

Lint 

43. Demo 2 — Failing Unit Test

Temporarily change a correct assertion:

```
expect(add(2,3)).toBe(5);
```

to:

```
expect(add(2,3)).toBe(500);
```

Push the change.

The pipeline becomes:

```
Checkout ✓  
Install ✓  
Lint ✓  
Tests ✗  
Build SKIPPED  
Package SKIPPED  
Deploy SKIPPED
```

Jenkins can show something similar to:

```
Expected: 500  
Received: 5
```

This demonstrates the **fast feedback loop**.

The developer learns about the problem before the code reaches deployment.

44. Add a Real Feature

Fix the test again:

```
expect(add(2,3)).toBe(5);
```

Now add a new function:

```
function multiply(a, b) {  
  return a * b;  
}
```

Expose it through the application and add a test:

```
test("multiplies two numbers", () => {  
  expect(multiply(6,7)).toBe(42);  
});
```

Our test count increases:

```
7
↓
8
```

Commit and push:

```
git add .
git commit -m "Add multiply feature"
git push
```

The feature branch pipeline becomes:

Lint	✓
Tests	✓
Build	✓
Artifact	✓
Deploy	⏸ SKIPPED

Why is deployment skipped?

Because the branch is:

```
feature/ci-demo
```

while our Jenkinsfile says:

```
when {
    branch 'main'
}
```

Only `main` is allowed to deploy.

45. Production Remains Untouched

Even though the feature branch passed all validations, the existing `main` deployment remains active.

Feature Branch

↓

Lint ☒
Tests ☒
Build ☒
Artifact ☒

↓

No Deployment

This separation is important.

Passing CI does not automatically mean the code has permission to modify the deployment environment.

46. Merge Into `main`

Merge:

feature/ci-demo

↓

main

This can be done through a GitHub Pull Request or locally.

Example:

```
git checkout main
git pull
git merge feature/ci-demo
git push
```

Jenkins detects that `main` changed.

Now:

Checkout	✓
Install	✓
Lint	✓
8 Tests	✓
Build	✓
Artifact	✓
Deploy	✓
Smoke Test	✓

The new application version is now deployed.

This is the transition from:

```
CI
↓
Validate and Build

to

CD
↓
Deploy Validated Software
```

Part 6 — Repeat the Same Idea With Spring Boot

The most important lesson is that **CI/CD concepts are not tied to Node.js**.

We can perform essentially the same workflow with a Java/Spring Boot application.

47. Create a Small Spring Boot Project

Using Spring Initializr:

```
Project: Maven
Language: Java
Java: 21
```

Packaging: Jar
Dependency: Spring Web

Project name:

jenkins-spring-demo

Keep the application deliberately small.

48. Calculator Service

Example:

```
@Service
public class CalculatorService {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public double discount(double price, double percentage) {
        if (price < 0 || percentage < 0 || percentage > 100)
        {
            throw new IllegalArgumentException("Invalid input");
        }

        return price - (price * percentage / 100);
    }
}
```

```
}  
}
```

49. Controller

```
@RestController  
@RequestMapping("/api")  
public class DemoController {  
  
    private final CalculatorService calculatorService;  
  
    public DemoController(CalculatorService calculatorService)  
    {  
        this.calculatorService = calculatorService;  
    }  
  
    @GetMapping("/health")  
    public Map<String, String> health() {  
  
        return Map.of(  
            "status", "UP",  
            "service", "jenkins-spring-demo",  
            "version",  
            System.getenv().getOrDefault("APP_VERSION",  
            "dev")  
        );  
    }  
  
    @GetMapping("/add")  
    public Map<String, Integer> add(  
        @RequestParam int a,  
        @RequestParam int b) {  
  
        return Map.of(  

```

```
        "result",  
        calculatorService.add(a, b)  
    );  
}  
}
```

50. Application Ports

Jenkins already uses:

```
8080
```

So configure Spring Boot to use:

```
server.port=8081
```

Our applications now use:

```
Jenkins    → 8080  
Spring Boot → 8081  
Node.js    → 3000
```

51. Spring Boot Unit Tests

Example:

```
class CalculatorServiceTest {  
  
    private final CalculatorService service =  
        new CalculatorService();  
  
    @Test
```

```

void shouldAddNumbers() {
    assertEquals(5, service.add(2, 3));
}

@Test
void shouldSubtractNumbers() {
    assertEquals(6, service.subtract(10, 4));
}

@Test
void shouldCalculateDiscount() {
    assertEquals(
        900,
        service.discount(1000, 10)
    );
}

@Test
void shouldRejectInvalidDiscount() {
    assertThrows(
        IllegalArgumentException.class,
        () -> service.discount(1000, 120)
    );
}
}

```

This gives us several automated tests before the application can be packaged or deployed.

52. Test Spring Boot Manually

Run tests:

```
./mvnw test
```

Package the application:

```
./mvnw package
```

Check:

```
ls -lh target/
```

We should get a JAR similar to:

```
jenkins-spring-demo-0.0.1-SNAPSHOT.jar
```

This `.jar` is our Spring Boot build artifact.

53. Make Maven Wrapper Executable

On the development machine:

```
chmod +x mvnw
```

This allows Jenkins to execute:

```
./mvnw
```

54. Spring Boot Jenkins Pipeline

The pipeline is very similar to the Node.js version:

```
pipeline {  
    agent any  
    options {
```

```

    skipDefaultCheckout(true)
}

stages {

    stage('Checkout') {
        steps {
            checkout scm
        }
    }

    stage('Compile') {
        steps {
            sh './mvnw -B -DskipTests compile'
        }
    }

    stage('Tests') {
        steps {
            sh './mvnw -B test'
        }

        post {
            always {
                junit 'target/surefire-reports/*.xml'
            }
        }
    }

    stage('Package') {
        steps {
            sh './mvnw -B -DskipTests package'
            archiveArtifacts 'target/*.jar'
        }
    }
}

```

```

stage('Deploy') {

    when {
        branch 'main'
    }

    steps {
        // Deploy JAR
    }
}

stage('Smoke Test') {

    when {
        branch 'main'
    }

    steps {
        sh '''
            sleep 5
            curl -f http://127.0.0.1:8081/api/health
        '''
    }
}

post {

    success {
        echo 'SPRING PIPELINE SUCCESSFUL'
    }

    failure {
        echo 'SPRING PIPELINE FAILED'
    }
}

```

```
}  
}
```

55. Understanding the Spring Pipeline

The stages are:

```
Checkout  
↓  
Compile  
↓  
Tests  
↓  
Package  
↓  
Archive JAR  
↓  
Deploy  
↓  
Smoke Test
```

The concepts are exactly the same as before.

Only the build tools and artifact format have changed.

56. Node.js vs Spring Boot

Node.js	Spring Boot
<code>npm ci</code>	Maven resolves dependencies
ESLint	Compilation/static checks
Jest	JUnit
<code>npm run build</code>	<code>mvn package</code>
<code>.tar.gz</code>	<code>.jar</code>
<code>node dist/server.js</code>	<code>java -jar app.jar</code>

The CI/CD principles remain the same:

```
Source Code
  ↓
Validation
  ↓
Automated Tests
  ↓
Build
  ↓
Artifact
  ↓
Deployment
  ↓
Verification
```

57. Push Spring Boot Project to GitHub

Create repository:

```
jenkins-spring-demo
```

Initialize:

```
git init
git add .
git commit -m "Create Spring Jenkins demo"
git branch -M main
```

Add remote:

```
git remote add origin \
https://github.com/YOUR_USERNAME/jenkins-spring-demo.git
```

Push:

```
git push -u origin main
```

58. Create the Spring Multibranch Pipeline

Inside Jenkins:

```
New Item
↓
Multibranch Pipeline
```

Name:

```
jenkins-spring-demo
```

Add the Git repository as the branch source.

Jenkins discovers `main` and executes:

```
Checkout
Compile
Tests
Package
Deploy
Smoke Test
```

59. Verify the JAR Artifact

Inside the Jenkins build, the generated artifact should appear under:

```
Artifacts
```

Example:

```
jenkins-spring-demo-0.0.1-SNAPSHOT.jar
```

Today:

```
Jenkins
  ↓
Stores JAR
```

A more production-oriented setup may later become:

```
Jenkins
  ↓
Nexus / Artifactory
  ↓
Deployment Environment
```

60. Verify Spring Boot Deployment

From Ubuntu:

```
curl http://127.0.0.1:8081/api/health
```

Example:

```
{
  "status": "UP",
  "service": "jenkins-spring-demo",
  "version": "1"
}
```

Test the API:

```
curl "http://127.0.0.1:8081/api/add?a=7&b=5"
```

Expected:

```
{
  "result": 12
}
```

```
}
```

From the Mac:

```
curl http://VM_IP:8081/api/health
```

Jenkins has now deployed an application built using a completely different technology stack.

61. Final Mental Model

Jenkins is not primarily about knowing how to run `npm`, Maven, Java, or Node.js.

Its job is to **orchestrate a software delivery process**.

```
Developer pushes code
    ↓
Repository changes
    ↓
Jenkins detects change
    ↓
Checkout
    ↓
Quality Gates
    ↓
Automated Tests
    ↓
Build
    ↓
Versioned Artifact
    ↓
Deployment Rules
    ↓
Deployment
    ↓
Smoke Test
```

If anything fails:

```
Failure
    ↓
Pipeline Stops
```

```
↓  
Later Stages Are Blocked  
↓  
Bad Build Is Not Deployed
```

If everything succeeds on a feature branch:

```
Validate  
↓  
Test  
↓  
Build  
↓  
Artifact  
↓  
No Production Deployment
```

Once the validated code reaches `main`:

```
main  
↓  
CI Passes  
↓  
Artifact Created  
↓  
Deployment Allowed  
↓  
Smoke Test  
↓  
New Version Running
```

That is the core idea behind the complete **CI/CD pipeline with Jenkins**.